

Managed IPsec VPN Service

Requirements Specification — the Catalyst 8000v IPsec VTI + eBGP lab, its Nautobot source of truth, the Network-as-Code pipeline, the VPN Provisioning Portal, compliance, monitoring and tests

Project	github.com/dcantor/cat8000v-ipsec (with lab-portal for the shared monitoring)
Version	1.0 — 2026-09-22 — derived from the delivered system: every requirement below is implemented and, where stated, verified by the Robot Framework suites (60 tests in 11 suites)
Provider / customers	ACME Networks sells the managed VPN service and owns the headends; every branch is a customer
Priority key	MUST mandatory SHOULD expected MAY optional / convenience

Contents

1. Purpose, scope and system context	13. Portal: access control, sign-in, audit
2. Stakeholders, roles and glossary	14. Portal: provisioning and day-2 operations (runs)
3. Network service requirements (topology, tunnels, routing, underlay)	15. Portal: pages (Provision, Jobs and a page per job, Inventory, Branches, router page, Firewalls, Tools, Audit)
4. Source of truth: the Nautobot model	16. Configuration compliance, remediation, scheduling and drift history
5. The intent document	17. REST API
6. Network-as-Code pipeline (render, plan, staged apply, convergence)	18. Monitoring, metrics and alerting
7. Firewalls	19. Verification: the Robot Framework suites
8. IKE authentication: pre-shared keys and certificates (PKI)	20. Lab infrastructure and operations (lab.sh)
9. Customers, provider and ACME design patterns	21. Documentation and demos
10. LAN hosts and the ping mesh	22. Non-functional requirements
11. Internet breakout	23. Constraints, assumptions and known limits
12. Data-centre interconnect: an acquired company, overlapping addresses, NAT and DNS	24. Traceability matrix

1. Purpose, scope and system context

The system provides a **managed hub-and-spoke IPsec VPN service**: a provider (ACME Networks) operates regional headends; each customer branch is a Catalyst 8000v spoke with IPsec VTIs to two or more headends and eBGP over them, so every site LAN reaches every other site LAN through the encrypted tunnels, and reaches the internet through the nearest headend's firewall. Everything about the service is described once (the intent), stored in **Nautobot** as the source of truth, rendered into **Network-as-Code** data and pushed to the routers by **Terraform**, checked by **Golden Config** compliance, proven by **Robot Framework** tests, and operated from a **web portal** with a REST API. The lab realises this on one KVM host with libvirt VMs, but the requirements describe the service, not the host.

Component	Responsibility	Where
Routers	3 C8000v headends (east / central / west), 4+ C8000v spokes (branches); IOS-XE 17.x, RESTCONF, SSH	libvirt VMs, OOB network 10.2.0.0/24

Firewalls	one VyOS firewall in front of each headend: the spokes' WAN links land on it; forward-filter policy, NAT breakout, syslog	libvirt VMs
LAN hosts	one Alpine Linux VM behind every router (its site LAN); the ping-mesh endpoints	libvirt VMs, 256 MiB
Nautobot 3.2 (+ bgp-models, golden-config, device-onboarding)	source of truth: devices, interfaces, cables, prefixes, VPN app, BGP, tenants, custom fields, config contexts, Golden Config	NMS VM 10.0.0.10 (shared with the SRv6 lab)
Gitea	Golden Config backup / intended / template repositories; OIDC provider for the portal	NMS VM
Network-as-Code	<code>netascode/nac-iosxe</code> module, <code>CiscoDevNet/iosxe</code> provider 0.15; data rendered from Nautobot	<code>nac/</code> , Terraform state on the lab host
Portal	FastAPI + single-page UI on :8090: provisioning, day-2 operations, inventory, branches, compliance, audit, metrics	lab host, <code>systemd --user lab-webapp</code>
Monitoring	Prometheus, VictoriaMetrics, VictoriaLogs, vmalert, Grafana (lab-portal/monitoring)	NMS VM, Docker Compose
Tests	Robot Framework suites 01–11; results per run with config backups and diffs	<code>tests/</code> , <code>results/</code>

2. Stakeholders, roles and glossary

Role	Needs
Viewer	read everything: inventory, branches, compliance, audit, configurations (secrets redacted)
Operator	provision spokes and headends, deploy the intent, change authentication, rotate keys, renew certificates, re-home, remediate drift, run Golden Config and tests
Approver	everything an operator can, plus decommissioning a spoke and managing portal users
Scheduler (system)	runs the periodic Golden Config compliance check as its own audited user
Customer (ACME account)	a company owning one branch; identified in Nautobot as a tenant with address, industry, account, service tier, contract and design pattern

Term	Meaning
Intent	<code>lab-intent.json</code> : the whole service in one document (site, VPN, profile, regions, devices, links, tunnels, internet, provider, customers)
Headend / hub	a provider router terminating spoke tunnels; one firewall in front of it; originates the default route
Spoke / branch	a customer router with a tunnel to at least two headends
Design pattern	ACME's attachment type of a branch, derived from its tunnel count: ACME-SH (1), ACME-DH (2), ACME-MH (3+); ACME-HE for headends
Run	an asynchronous portal pipeline (deploy, plan, test, spoke, hub, remove, rotate, rehome, renew, auth, golden, remediate, reapply) with steps, log, status and a test report
Golden Config	Nautobot app: backup → intended (Jinja from the model) → compliance per feature; remediation lines per non-compliant feature

Staged apply	<code>tools/nac_apply.py</code> : creates → in-place updates → the full plan, each from a saved plan (optionally for one device)
--------------	--

3. Network service requirements

ID	Prio	Requirement	Acceptance criteria · verified by
NET-01	MUST	Every branch is connected to at least two headends by point-to-point IPsec VTIs (<code>tunnel mode ipsec ipv4</code> , one TunnelN per (headend, spoke)), sourced from the spoke's WAN link to that headend's firewall.	tunnels up/up at both ends, tunnel subnet reachable, exactly one IKEv2 session per spoke tunnel on each hub · suite 03; the wizard refuses fewer than two headends · suite 10 / <code>spokes.validate</code>
NET-02	MUST	IKEv2 with the modelled proposal (encryption / integrity / DH group / lifetimes from the VPN profile) and IPsec transform set; DPD on; MSS clamping and MTU on the VTIs.	IKEv2 SAs READY with the modelled algorithms; traffic encrypted with the modelled transform · suite 03
NET-03	MUST	eBGP over the VTIs : one AS per site, every hub peers with each of its spokes, each site advertises its LAN (and loopback) so every site LAN reaches every other through a headend.	sessions Established, every spoke learns every other site, spoke-to-spoke paths go through a hub, LANs ping across tunnels · suite 04
NET-04	MUST	WAN underlay : each spoke has a /30 point-to-point link per headend, landing on the headend's firewall (spoke port ↔ firewall port); the headend's WAN link is behind the same firewall; LLDP on the links.	interfaces up with the modelled addresses, headend WAN reachable through the firewall, LLDP shows the firewall · suite 02
NET-05	MUST	Management plane : a dedicated OOB port in <code>Mgmt-vrf</code> with a default route, SSH and RESTCONF, AAA, VTY hardening and a management ACL from the NaC baseline; the security license level.	routers reachable over OOB by SSH and RESTCONF; baseline present · suite 01
NET-06	MUST	Each router has a site LAN /24 on its LAN port (.1; the LAN host .2) and a router-id loopback /32; site LANs are unique; a /24 is required by the classful BGP network statement.	intent validation rejects non-/24 or duplicate LANs · <code>intent.validate</code> ; loopbacks carry router-id · suite 02
NET-07	SHOULD	Headend capacity is bounded: a tunnel-slot limit per headend and the bandwidth of the firewall in front of it (each tunnel commits a fixed share); provisioning is refused when either would be exceeded.	capacity modelled in Nautobot, enforced by <code>spokes.validate_links</code> , shown on the Inventory page; suites 07 and 10

4. Source of truth: the Nautobot model

ID	Prio	Requirement	Acceptance criteria · verified by
SOT-01	MUST	Nautobot is the single source of truth : nothing about the topology is hard-coded in templates; the NaC renderer and the Golden Config template read only Nautobot objects (GraphQL saved query <code>nac-c8000v-ipsec-model</code>).	NaC data rendered from Nautobot equals the committed file; <code>render --check clean</code> · suite 06

S0T-02	MUST	Routers are onboarded (Device Onboarding app: model, serial, platform, management IP) and carry role (<code>vpn-hub</code> / <code>vpn-spoke</code> / <code>vpn-firewall</code> / <code>lan-host</code>), platform, secrets group, status, location, tenant and comments.	suite 06 "Routers exist in Nautobot..."
S0T-03	MUST	Physical model: every router port with description and MAC; cables for each WAN link (spoke port ↔ firewall port, headend ↔ firewall), the LAN link to the host; unwired ports disabled and address-free; loopbacks and Tunnel interfaces of type tunnel; prefixes with roles (<code>oob-management</code> , <code>wan-p2p</code> , <code>vpn-tunnel</code> , <code>site-lan</code> , <code>loopback</code>); the <code>bgp:advertise</code> tag on advertised prefixes. Ports cabled by another lab are left alone.	suite 06 "WAN links are modelled as cables..."; seed idempotent (a second run reports no changes)
S0T-04	MUST	VPN model in Nautobot's core VPN app: Phase 1 / Phase 2 policies → one VPN profile per IKE authentication method (<code>VPN-IPSEC</code> , <code>VPN-IPSEC-PSK</code>) → the VPN → one VPN tunnel per (headend, spoke) with hub / spoke endpoints (source interface + address, tunnel interface, protected prefixes); Cisco object names in the profile's <code>extra_options</code> .	tunnel destination equals the far endpoint's source address; suite 06 "The VPN is modelled..."
S0T-05	MUST	BGP model (nautobot-bgp-models): one autonomous system per site, a routing instance per router, an eBGP peering per tunnel with both endpoints.	suite 06 "BGP model..." vs live sessions
S0T-06	MUST	Location hierarchy lab site → Region (ordered geographically) → Branch / HQ, with <code>site_code</code> and <code>contact</code> custom fields, coordinates from the site's city (for the map) and the physical address.	suite 06 "Locations form the hierarchy..."
S0T-07	MUST	Custom fields carry what the core model lacks: <code>vpn_tunnel_capacity</code> , <code>firewall_bandwidth_mbps</code> , per-tunnel <code>psk_fingerprint</code> / <code>psk_rotated</code> , per-router <code>cert_serial</code> / <code>cert_subject</code> / <code>cert_expires</code> / <code>cert_renewed</code> , and the ACME fields of §9. A newly created field is saved a second time so Nautobot's field cache accepts values (<code>nudge_cf</code>).	suites 06 and 08 read them back
S0T-08	MUST	Config contexts hold the operational policy: OOB VRF / gateway / ACL and domain, the firewall policy (<code>firewall.forward</code> , management syslog / LLDP), and <code>internet</code> (enabled, uplink, NAT sources, per-spoke preference).	rendered firewalls and breakout match the context · suites 07 and 11
S0T-09	MUST	The seed is idempotent and complete : <code>./lab.sh nautobot seed</code> creates or updates every object from the intent, reports each change, removes what the intent no longer has (dropped links, retired tenants, the legacy <code>Loopback10</code>), and never touches pre-shared keys (only fingerprints).	a second seed reports "0 changes"; secrets absent from Nautobot · suite 06
S0T-10	MUST	Golden Config is configured from code: settings, scope (the IOS-XE routers of the site), the Jinja template <code>c8000v-ipsec.j2</code> in Gitea, compliance features and rules (BGP, BGP policy, banner, tunnels, IKEv2/IPsec incl. the trustpoint and certificate map, loopbacks, management ACL / interface, NTP, SNMP, static routes, syslog, VLANs, VRFs, WAN interfaces) with remediation enabled; the SoT query is patched with what the template needs.	backups in Gitea, 100 % of compliance rows compliant after a deploy · suite 06 "Golden Config...", suite 10

5. The intent document

ID	Prio	Requirement	Acceptance criteria · verified by
INT-01	MUST	One JSON document describes the whole service: site, VPN service (name, change ticket, owner), profile (IKE / IPsec / DPD, IKE authentication default, PKI settings, Cisco names), regions, devices (routers, firewalls, hosts with identity, LAN, region, site, city / coordinates, PSK, per-spoke authentication, customer, address), links, tunnels, OOB, capacity, internet, provider.	schema in <code>webapp/schemas.py</code> (Swagger); <code>intent.validate</code>
INT-02	MUST	The intent is validated before anything runs: unique hostnames / router-ids / LANs / AS numbers, IPv4 addresses, /24 LANs, a firewall per headend at most, every VM accounted for, the links equal to the physical wiring (in the renamed hostnames), each host wired to its router's LAN.	<code>POST /api/validate</code> returns the problems; the Deploy button is disabled while invalid
INT-03	MUST	Older documents are upgraded on load (multi-hub tunnels, per-spoke PSKs, regions and branches per device, capacity defaults, provider, generated customers and headend addresses), so the pipeline never depends on a hand-edit.	<code>intent.upgrade</code> ; every consumer calls <code>intent.load()</code>
INT-04	MUST	Pre-shared keys live only in the intent (and NaC variables); Nautobot and the audit log see fingerprints; the portal redacts keys for viewers.	suite 06 "Every PSK spoke uses its own key..."; audit redaction in <code>auth.redact</code>
INT-05	SHOULD	The intent can be initialised from the lab's wiring (<code>./lab.sh intent init</code>) and inspected / validated from the CLI.	<code>lab.sh intent init validate show</code>

6. Network-as-Code pipeline

ID	Prio	Requirement	Acceptance criteria · verified by
NAC-01	MUST	Render: <code>nautobot/render_nac.py</code> builds <code>nac/data/devices.nac.yaml</code> and <code>device_groups.nac.yaml</code> from Nautobot (GraphQL): interfaces, loopbacks, tunnels, crypto (one IKEv2 / IPsec profile per authentication method in use, keyrings only with PSK peers), BGP neighbours and address families, breakout policy, static routes, banner, management baseline; secrets come from group variables.	<code>render --check</code> equals the committed data · suite 06
NAC-02	MUST	What the iosxe provider cannot express is rendered as <code>iosxe_cli</code> templates (rsa-sig / certificate map / trustpoint on the IKEv2 profile, MSS clamping) on top of the native resources.	certificate tunnels authenticate with rsa-sig · suite 08
NAC-03	MUST	Plan is a first-class dry run (<code>-detailed-exitcode</code>): the portal's Dry run and every deploy report "no changes" or the plan summary before applying.	terraform plan reports no drift after a deploy · suite 05

NAC-04	MUST	Staged apply (<code>tools/nac_apply.py</code>): creates first, then in-place updates, then the full plan — each stage from a saved plan file — because the module has no tunnel → profile dependency and IOS refuses to delete an in-use profile or keyring. The CLI templates only go in the last stage. <code>--device NAME</code> restricts every stage to one router's resources; <code>--dry-run</code> lists the changes.	auth switches and re-homes apply without "Device refused" errors · suites 08 and 10 (reapply)
NAC-05	MUST	After an apply the running configuration is saved on every router; a post-apply re-plan re-asserts once when IOS-XE's DMI re-sync elides values (convergence).	<code>lab.sh nac apply</code> saves; <code>Run.do_apply</code> convergence
NAC-06	MUST	A decommissioned spoke is forgotten in the Terraform state (its resources removed) before the hub-side tunnel, neighbour and addresses are destroyed by the next apply.	remove workflow · <code>spokes.forget_in_terraform</code>

7. Firewalls

ID	Prio	Requirement	Acceptance criteria · verified by
FW-01	MUST	Exactly one VyOS firewall in front of each headend, reachable over OOB; its interfaces (addresses, descriptions, hw-id) come from the Nautobot model and cables; unwired ports are disabled.	suite 07 "Each headend has exactly one firewall...", "Firewall interfaces carry the modelled addresses..."
FW-02	MUST	The forward-filter policy is rendered from the config context: established / related, IKEv2 (udp 500 / 4500) and ESP only between the modelled peers (address groups of the headend's and the cabled spokes' WAN addresses, both directions), ICMP, a logged drop for everything else; optional logging of accepted first packets.	only IKEv2 / ESP / ICMP cross; non-VPN traffic dropped · suite 07; rendered config equals running config (<code>vyos --check</code>)
FW-03	MUST	The managed sections are replaced inside one commit so VyOS applies only the difference; the push reports "committed" or "no changes" and fails on any rejected line.	<code>render_vyos.push</code>
FW-04	MUST	Firewalls ship their kernel firewall log by remote syslog to VictoriaLogs (host / port / protocol / level from the context); LLDP enabled.	suite 07 "The firewalls ship their firewall log..."; dashboard panels and log alerts (§18)
FW-05	SHOULD	The portal's Firewalls page shows each firewall's interfaces, rules with hit counters, the flows and the drops it logged, collected over SSH / VictoriaLogs; a router's page shows the rules and log lines touching that router.	<code>GET /api/firewalls</code> , <code>GET /api/branch/{name}</code>

8. IKE authentication: pre-shared keys and certificates

ID	Prio	Requirement	Acceptance criteria · verified by
----	------	-------------	-----------------------------------

PKI-01	MUST	Each branch chooses pre-shared key or certificate — at provisioning (wizard) or later (Change auth) — with the profile's <code>ike.authentication</code> as the default. Headends serve both: one IKEv2 / IPsec profile per method in use; the PSK profile matches its peers' /32 addresses, the certificate profile matches the certificate map.	suite 08 (every test), suite 03 "IKEv2 SAs READY ... each spoke's chosen method"
PKI-02	MUST	A lab CA (<code>pki/ca.py</code> , key never committed) issues each router a certificate (CN = fqdn, configurable key size, validity, renew-before); <code>nautobot/pki.py</code> enrolls over SSH (trustpoint with the CA fingerprint pinned, key pair, CSR signed and imported), retires routers that no longer need one, and records serial / subject / expiry / renewal in Nautobot.	CA pinned on every certificate router; certificates issued to the router's own name and not near expiry; Nautobot records them · suite 08
PKI-03	MUST	Per-spoke keys: every PSK spoke uses its own key on all of its tunnels; each headend keys per PSK spoke and holds no key for a certificate spoke ; keys can be rotated (new key → intent → Nautobot fingerprint / date → NaC → apply → SAs cleared → verified).	suite 06 "Every PSK spoke uses its own key..."; rotate run · <code>Run.do_rot_*</code>
PKI-04	MUST	After an apply that changed authentication, post-apply verification re-enables VTIs IOS shut when their protection profile was swapped, removes keyrings on key-less routers, clears SAs still authenticated the old way, and waits until every tunnel is READY with the modelled method (RSA / PSK) — otherwise the run fails.	<code>pki.py --post-apply</code> ; suite 08 "Certificate tunnels authenticate with rsa-sig..."
PKI-05	MUST	Renewal from the portal (or when within renew-before days at any pki step): a new CSR, signed and imported, Nautobot updated, the router's SAs cleared, every tunnel back READY with RSA.	suite 08 "The portal renews a spoke's certificate..."
PKI-06	SHOULD	Certificate expiry is exported as a metric and alerted (30 days warning, expired critical); the portal shows days left per router.	<code>lab_cert_not_after_seconds</code> ; alerts <code>CertificateExpiringSoon</code> / <code>CertificateExpired</code>

9. Customers, provider and ACME design patterns

ID	Prio	Requirement	Acceptance criteria · verified by
CUST-01	MUST	The provider (ACME Networks) and every customer are Nautobot tenants : tenant group <i>Service provider</i> for ACME (headends, firewalls, HQ sites, headend hosts) and <i>Customers</i> with one tenant per company on the branch's router, LAN host and location.	suite 06 "Every branch is a customer of ACME..."
CUST-02	MUST	Each customer carries company, street address (the location's physical address), industry, ACME account id, service tier (Bronze / Silver / Gold) and contract start — stored on the spoke in the intent, generated deterministically when missing, proposed by the wizard and editable; company names unique, tier from the catalogue, ISO date.	custom fields <code>acme_account_id</code> , <code>acme_service_tier</code> , <code>acme_industry</code> , <code>acme_contract_start</code> on the tenant · suite 06; <code>spokes.validate</code>

CUST-03	MUST	The ACME design pattern of a branch is derived from its number of tunnels — ACME-SH single headend (1), ACME-DH dual headend, resilient (2), ACME-MH multi headend, any-region (3+); headends ACME-HE — and written to <code>acme_design_pattern</code> (select) and <code>acme_pattern_tunnels</code> on the router and its location, so re-homing changes the pattern at the next seed.	suite 06 checks the pattern against the tunnel count in Nautobot and on the portal
CUST-04	MUST	The portal exposes and filters this metadata: the Branches page shows customer / owner, address, pattern and tier and filters by free text, role, region, pattern, tier, industry, IKE auth, health and VM state; a router's page opens with the customer box and a link to the tenant.	<code>GET /api/branches</code> (customer, pattern, provider, patterns with counts, tiers, regions) · suite 06
CUST-05	SHOULD	Removing a branch removes its tenant once no device is left with it.	<code>spokes.remove_from_nautobot</code>

10. LAN hosts and the ping mesh

ID	Prio	Requirement	Acceptance criteria · verified by
HOST-01	MUST	A small Alpine VM sits behind every router on its site LAN (.2, gateway .1 on the router's LAN port), managed over OOB, modelled in Nautobot (device, interfaces, cable, tenant) and in the intent.	suite 09 "The LAN hosts...", suite 06
HOST-02	MUST	Full ping mesh: every host pings every other host (through the tunnels), its own gateway and the internet (1.1.1.1); results per pair with loss and latency; from the CLI (<code>./lab.sh hosts</code>) and the portal (<code>GET /api/hosts?ping=true</code>).	suite 09 "full host-to-host ping mesh"; suite 11 internet
HOST-03	SHOULD	The Inventory page shows the mesh green / red with a live mode (one probe per pair every 5 s, round-trip time in the cell) and columns for the router itself and the internet.	<code>GET /api/hosts?ping=true&live=true</code>

11. Internet breakout

ID	Prio	Requirement	Acceptance criteria · verified by
INET-01	MUST	When enabled in the intent, each firewall NATs the site LANs (network group) out of its uplink (DHCP on the libvirt NAT network), admits only the LAN → uplink direction, and routes the site LANs back to its headend; headends hold a static default via the firewall and originate a default to their spokes.	NAT, return routes and default origination · suite 11; rendered config equals running · suite 07
INET-02	MUST	Each spoke prefers the nearest headend (regions are ordered; preference by distance from the spoke's region) through inbound route-maps setting local preference on the default route; the preference is in the Nautobot config context.	suite 11 "Every spoke prefers the nearest headend's default..."; every host reaches the internet through its region's headend

INET-03	SHOULD	The portal shows the breakout state on the Provision card and each branch's preference (with the live default route) on its page; the monitoring alerts <code>InternetBreakoutDown</code> .	<code>intent.internet()</code> , <code>GET /api/branch/{name}</code>
---------	--------	---	--

12. Data-centre interconnect: an acquired company, overlapping addresses, NAT and DNS

ID	Prio	Requirement	Acceptance criteria
DCI-01	MUST	A data-centre interconnect hangs off one headend (west-headend Gi3 → DCI → ACME-acquisition) on plain routed /30s with no IPsec : ACME owns the DCI (tenant <i>ACME Networks</i> , role <code>vpn-dci</code>), the acquired company is a customer tenant of its own on a <code>partner-edge</code> router. Devices, cables, prefixes, peerings and tenants are modelled in Nautobot and rendered by the same NaC pipeline and Golden Config template as everything else.	suite 12 "The DCI chain is modelled in Nautobot...", "The interconnect links are up ... no IPsec anywhere on them"
DCI-02	MUST	eBGP per hop (65206 ↔ 65208 ↔ 65209) from the same nautobot-bgp-models peerings the tunnels use; the acquisition originates its site LAN and a service prefix on a loopback, which every branch learns through one of its headends and can reach from its own LAN address; the host behind the acquisition joins the ping mesh and breaks out to the internet through the headend.	suite 12 "eBGP is Established on every hop...", "The acquisition originates its prefixes...", "A branch reaches the acquisition's LAN...", "The host behind the acquisition is in the mesh..."
DCI-03	MUST	Overlapping address space is translated, not renumbered : the acquired company keeps a prefix ACME already uses (its own Nautobot namespace), and the DCI runs a twice-NAT — <code>ip nat inside source static network (inside local → inside global)</code> and <code>ip nat outside source static network (outside global → outside local)</code> — with <code>ip nat inside / outside</code> on the two interconnect ports. A prefix-list <code>OVERLAP</code> and route-map <code>NO-OVERLAP</code> filter the colliding prefix in and out towards ACME and out towards the acquisition, and the translated ranges are advertised instead (a <code>Null0</code> route for the inside-global range, a route towards ACME for the outside-local one).	suite 12 "The overlapping prefix exists on both sides ... neither side learns the other's copy", "The DCI translates in both directions..."
DCI-04	MUST	DNS fix-up in both directions : each side answers for its own zone (<code>ip dns server + ip host</code> records rendered from the intent), the far side resolves through the DCI from an address the NAT translates, and the ALG rewrites the A records in flight so what a client resolves is what it can reach. A LAN host's resolver comes from the intent (<code>dns_client</code> , written by cloud-init; the instance-id follows a hash of the seed so a changed resolver is applied on the next boot).	suite 12 "DNS is fixed up on the way through the NAT...", "DNS fix-up scales and works from either side..." and "The fix-up is visible on the wire..." — a packet capture on both of the DCI's interfaces shows the same answer carrying the real address inside and its translated form outside (<code>tools/dns_capture.py</code> , .pcap kept with the run)

DCI-05	MUST	Scale: a <code>scale</code> block expands to 1000 overlapping /24s on both sides — 2002 static translations on the DCI — while BGP carries only three aggregates; each side holds its thousand addresses as secondary addresses on one loopback, and the renderer splits CLI templates into 400-line chunks (IOS-XE's CLI RPC rejects a larger payload). Each zone carries 1000 records over the same set.	suite 12 "NAT scales: a thousand overlapping prefixes..." (reached 1000/1000, no drops, aggregates only) and "DNS fix-up scales..." (resolved+reached 1000/1000)
DCI-06	MUST	The NAT and the DNS are under configuration compliance like everything else: two Golden Config features (NAT (DCI), DNS) whose intended configuration is rendered from the same config context — including the full scale set — so a hand-edit anywhere in it shows up as drift.	171/171 compliance rows compliant; suite 12 "Golden Config renders the DCI chain..."
DCI-07	SHOULD	The interconnect is visible in the portal: the DCI and the acquisition appear on both topology views (dashed, as non-IPsec links) and in the Branches table with their tenant and design pattern.	topology screenshots; suite 10 / suite 06

13. Portal: access control, sign-in, audit

ID	Prio	Requirement	Acceptance criteria · verified by
SEC-01	MUST	Roles viewer / operator / approver: reads need a viewer; writes an operator; removing a spoke and user management an approver; validation endpoints are open to viewers; a few read-only endpoints (metrics, pki, hosts, oidc) are open for the collectors and tests.	<code>auth.required_role</code> ; 401 / 403 with the required role; suite 10 checks role gates
SEC-02	MUST	Local users (salted script hashes in <code>users.json</code> , CLI to add users, session cookie, bearer tokens for automation) and single sign-on via OpenID Connect (authorization code + PKCE, JWKS-verified ID token, provider groups → role; Gitea in the lab).	suite 10 "Single sign-on is offered..."; <code>webapp/auth.py</code>
SEC-03	MUST	Audit trail : every login (local / SSO / failed), run start / resume / cancel, write and denied request is recorded with user, role, source address, path, mode, spec (secrets redacted: a PSK becomes its fingerprint, passwords and tokens are dropped) and the run id; readable on the Audit tab and <code>GET /api/audit</code> with filters.	<code>webapp/runs/audit.jsonl</code> ; suite 10
SEC-04	MUST	Secrets never reach Nautobot, the audit log or viewers' screens: router credentials come from environment / secrets group, PSKs appear only to operators, remediation lines and configurations are redacted for viewers.	<code>branch_config</code> redaction; <code>Run.do_rem_validate</code> log redaction
SEC-05	SHOULD	The UI adapts to the role: buttons the user cannot use are disabled with the required role as tooltip; a 401 opens the sign-in dialog.	<code>applyRoles()</code>

14. Portal: provisioning and day-2 operations (runs)

ID	Prio	Requirement	Acceptance criteria · verified by
RUN-01	MUST	Every change is an asynchronous run (<code>POST /api/runs</code> → id; <code>GET /api/runs/{id}</code> with incremental log) made of named steps with status, timing and a one-line summary; a run ends success / failed / cancelled / interrupted, carries a results directory and a parsed Robot test report when tests ran.	suite 10; run records under <code>webapp/runs/</code>
RUN-02	MUST	Runs execute one at a time in the order started (one Terraform state, one intent); a queued run shows its place in line and the run it waits for and can be cancelled; a portal restart marks in-flight runs interrupted.	suite 10 "runs queue instead of colliding..."; <code>DELETE /api/runs/{id}</code>
RUN-03	MUST	A failed or interrupted run can be resumed from the failed step, keeping the successful steps and the original starter; the resume is audited.	<code>POST /api/runs/{id}/resume</code>

RUN-04	MUST	deploy: validate → save intent → Nautobot seed → render → pki → firewalls → plan → staged apply → post-apply verification → Golden Config → tests (Golden Config and tests optional). plan: the same up to terraform plan, nothing pushed. test: the Robot suites only.	<code>Run.plan()</code> ; suites 05 / 06 / 10
RUN-05	MUST	spoke: provision a new branch end to end — validate the allocation → register in <code>lab.conf</code> + day-0 config → create and boot the VM → bootstrap (day-0, license reload, RESTCONF) → onboard into Nautobot → add to the intent (links, tunnels, BGP, customer) → seed → render → pki → firewalls → plan → apply → verify → Golden Config → tests. Hub and spoke are configured in one Terraform run.	wizard + <code>GET /api/spokes/suggest</code> , <code>POST /api/spokes/validate</code> ; suite 10 exercises validation
RUN-06	MUST	hub: provision a new headend with a link and tunnel to every selected spoke, the same pipeline.	<code>GET /api/hubs/suggest</code> , <code>HubSpec</code>
RUN-07	MUST	remove (approver): show what is released and what each headend loses → power off → Nautobot clean-up (tunnel, endpoints, BGP, device, addresses, prefixes, AS, tenant) → intent and <code>lab.conf</code> → Terraform state → apply on the headends → delete the VM (disk optional) → Golden Config → tests.	<code>GET /api/spokes/{name}/removal</code> ; <code>spokes.remove_*</code>
RUN-08	MUST	rehome: change a running branch's set of headends (at least two): new links / tunnels allocated, the VM redefined and rebooted for the new NIC, dropped ones removed from intent, <code>lab.conf</code> and Nautobot, terraform on hubs and spoke, verification that new tunnels are READY + Established and dropped ones gone.	<code>GET /api/spokes/{name}/rehome</code> ; suite 10 branch actions
RUN-09	MUST	rotate (PSK), auth (PSK ↔ certificate), renew (certificate) as described in §8, each with a plan / preview endpoint the dialog shows before starting.	suite 08 performs a real renewal and a real switch there and back
RUN-10	MUST	golden , remediate , reapply as described in §16.	suite 10 remediation tests
RUN-11	SHOULD	Every spec is validated before a run starts (422 with the list of problems) so a run never begins on bad input; suggestions (next free management IP, node index, ports, /30s, tunnel ids, router-id, LAN, AS, site, city, PSK, customer) are offered for every field.	<code>spokes.suggest / validate</code> ; suite 10

15. Portal: pages

ID	Prio	Requirement	Acceptance criteria · verified by
UI-01	MUST	Provision: the intent as a form (site, service, profile, crypto, PSK policy, internet breakout, capacity); the routers table with role, region, site, AS, router-id, LAN, IKE method, PSK (operators), certificate days left, comments, and the day-2 actions (Change auth, Rotate PSK, Renew cert, Re-home, Remove); Add spoke / Add headend wizards; Deploy / Dry run / Run tests; the run status panel (steps, log, test report) and the recent runs list.	screenshots and demo; suite 10

UI-02	MUST	Add-spoke wizard: step 1 identity, metadata, city (catalogue with coordinates, or any name with lat / lon), IKE authentication, PSK, customer block, headends (≥ 2); step 2 auto-suggested addressing, editable and re-validated; step 3 the review of what happens on the spoke and on each headend (WAN ports, tunnels, neighbours, Nautobot objects, design pattern, capacity after) → Provision.	SpokeSpec ; hub_changes
UI-03	MUST	Inventory: KPIs (Nautobot's VPN model joined with live IKEv2 / VTI / eBGP / ESP state from the headends), a rendered topology (headends on top, spokes below, one line per tunnel coloured by health, hover for details, click to open in Nautobot) and a map of the sites, headend capacity (tunnels, bandwidth, CPU / QFP / DRAM, the binding constraint), the per-tunnel report with CSV export, the LAN hosts and their ping mesh.	GET /api/vpn-inventory[.csv] ; suite 07 "Headend CPU is collected live..."
UI-04	MUST	Branches: every branch and headend with customer / owner, site and address, design pattern, tier, AS, LAN, VM state, tunnels up, IKE method, certificate, LAN host, free slots, last run; the filter bar of CUST-04; rows open the router's page.	GET /api/branches ; suite 10 "The Branches list..."
UI-05	MUST	Router page (#branch/<name>): customer / owner box, identity (site, router-id, LAN, WAN addresses, firewall, internet preference), authentication and certificate with the day-2 actions, tunnels with live state (and a headend's capacity line), firewall rules / flows / log touching it, its LAN host, the runs that involved it, live show commands from an allow-list (IKE, IPsec, BGP, routes, default, interfaces, PKI, platform, log; other commands refused), and the configuration: running config over SSH (re-read, download), Nautobot's intended config and last backup, compliance per feature with a running-vs-intended diff, the Gitea backup history with per-commit diffs, a dark / light toggle for the code panes.	GET /api/branch/{name} [/config /history /show/{what}] ; suite 10 "Every router has a page...", "... shows its configuration", "...history from Gitea"
UI-06	MUST	Jobs (#jobs): every run the portal has kept, newest first — what the job does, its target, its status, a progress bar with the step it is on , its duration (ticking while it runs), who started it, the change ticket and the test result; filters by status, by kind of job and a search over router / ticket / user; the list refreshes itself. A job's own page (#job/<id>): what the job does, KPIs (status, steps done, elapsed, who started it, tests passed), every step with its summary and duration, the live log with a follow toggle, the test report with links to the artefacts, Cancel for a queued job and Resume for a failed one. Starting anything anywhere in the portal opens that job's page, and every job id elsewhere links to it.	screenshots; GET /api/runs , GET /api/runs/{id}?since=
UI-07	MUST	Firewalls, Compliance (§16), Audit (§13), Tools (every lab system with URL and login, how to reach each device), the REST API (Swagger / ReDoc) from the navigation.	screenshots; suite 07 / 10

UI-08	SHOULD	Dark mode for the whole portal (CSS tokens; follows the OS setting until chosen; header toggle; remembered per browser).	screenshot portal-dark.png
UI-09	SHOULD	Every router name anywhere (tables, tunnels, compliance, history) links to its page; Nautobot objects link to Nautobot; the header shows the signed-in user and role.	branchLink()

16. Configuration compliance, remediation, scheduling and drift history

ID	Prio	Requirement	Acceptance criteria · verified by
CMP-01	MUST	The Compliance page shows Nautobot Golden Config's verdict: one row per router, one column per feature (✓ / ✗), KPIs (routers and rows compliant, features, last run, schedule), each device's last backup / intended / compliance dates; a cell shows the missing and extra lines, Nautobot's remediation lines and a running-vs-intended diff; non-compliant rows are listed and the first opened.	GET /api/compliance (cached 30 s, ?refresh); suite 10 "The Compliance page..."
CMP-02	MUST	Run Golden Config now (operator): backup every router → render the intended configuration → compliance, nothing pushed; the run fails when any row is non-compliant (so drift is visible as a failed run) and refreshes the report at its end.	run mode golden; suite 10
CMP-03	MUST	Remediate (operator): for a router's non-compliant features (one, or all), read the verdict again, push the remediation lines Nautobot computed (hier_config: no ... for what is extra, the missing lines themselves) in one configuration session, fail on any rejected line, save the configuration, then run Golden Config; refuse a compliant router or a feature without remediation lines.	run mode remediate; suite 10 "Drift on a router is detected..., remediated..."
CMP-04	MUST	Re-apply from the model (operator): re-assert the model on one router — NaC render → Terraform plan filtered to the router's resources → staged apply targeted at them (skipped when clean) → Golden Config.	run mode reapply, nac_apply.py --device; suite 10 "Re-apply from the model restores..."
CMP-05	MUST	A scheduled Golden Config run every <code>GOLDEN_INTERVAL_HOURS</code> (default 6; 0 disables), queued as the audited user <code>scheduler</code> , first due one interval after Nautobot's last compliance run (at the earliest two minutes after start-up), skipped and retried while another run is going or the headends are off; interval, next and last run exposed on the page and the API.	golden_schedule(); suite 10 asserts the schedule is enabled and in the future
CMP-06	MUST	Drift history : one snapshot per compliance run (per router: compliant, rows, drifted features) appended to <code>compliance-history.jsonl</code> whenever a new compliance date appears; the last 30 shown as dots per router and as a table (which routers drifted, when fixed); per-router series on its page and at GET /api/compliance/history?device=.	suite 10 checks the drift and its repair appear in the history

CMP-07	MUST	Compliance is exported to Prometheus (<code>lab_config_compliance_ok{device}</code> , <code>lab_config_noncompliant_features</code> , <code>lab_config_compliance_features</code> , <code>lab_config_compliance_last_run_timestamp_seconds</code> , <code>lab_config_compliance_interval_seconds</code> , <code>lab_config_compliance_error</code>) with alerts <code>ConfigDrift</code> (non-compliant 10 min) and <code>ConfigComplianceStale</code> (no run for twice the interval while the lab is up) and a dashboard row.	suite 10 reads the metric before and after remediation; lab-portal <code>alerts.yml</code>
CMP-08	MUST	Terraform-level compliance: after a deploy <code>terraform plan</code> reports no drift from the NaC data model.	suite 05

17. REST API

ID	Prio	Requirement	Acceptance criteria · verified by
API-01	MUST	Everything the UI does is available as a documented REST API (OpenAPI 3.1 at <code>/openapi.json</code> , Swagger UI at <code>/docs</code> , ReDoc at <code>/redoc</code>) with Pydantic schemas and examples; the UI uses nothing else.	tags intent, provisioning, runs, inventory, monitoring; suite 10 uses the API throughout
API-02	MUST	Endpoints: <code>/api/intent</code> , <code>/api/validate</code> , <code>/api/inventory</code> ; <code>/api/spokes/suggest</code> , <code>/api/spokes/validate</code> , <code>/api/hubs/suggest</code> , <code>/api/cities</code> , <code>/api/spokes/{name}/removal rehome rotation auth</code> , <code>/api/routers/{name}/renewal</code> ; <code>/api/runs</code> (list, start, get with <code>since</code> , <code>resume</code> , <code>cancel</code>); <code>/api/vpn-inventory[.csv]</code> , <code>/api/branches</code> , <code>/api/branch/{name}</code> [<code>/config</code>]/ <code>/history</code>]/ <code>show/{what}</code>], <code>/api/firewalls</code> , <code>/api/hosts</code> , <code>/api/pki</code> , <code>/api/compliance[/history]</code> , <code>/api/tools</code> , <code>/api/sd</code> ; <code>/api/login logout me oidc*</code> , <code>/api/users</code> , <code>/api/audit</code> ; <code>/metrics</code> .	<code>webapp/app.py</code>
API-03	MUST	Errors are explicit: 400 for an unknown mode, 401 / 403 with the required role, 404 for unknown routers / runs, 409 when a run cannot start, 422 with the list of problems for an invalid spec, 502 when a backing system (Gitea, Nautobot) fails.	suite 10 asserts 404 / 422 / 4xx cases
API-04	SHOULD	Expensive reads are cached with an explicit <code>refresh</code> (inventory live state, firewalls, configuration 60 s, show commands 20 s, compliance 30 s, host mesh 60 s / live 3 s) and report when they were generated.	<code>generated</code> field on the responses

18. Monitoring, metrics and alerting

ID	Prio	Requirement	Acceptance criteria · verified by
----	------	-------------	-----------------------------------

MON-01	MUST	The portal exposes <code>/metrics</code> : VMs running per node / role, per-tunnel health (2 up / 1 degraded / 0 down), IKE SA age, BGP prefixes, ESP counters and errors, VTI rates; per-headend tunnels / capacity / free slots / utilisation / CPU / QFP / DRAM / IKE sessions / bandwidth; certificate expiry and authentication per spoke; compliance (§16); the interconnect (static translations on the DCI against the model, the NAT table, drops per direction, the aggregates' presence in the routing table, records per DNS zone and the packet-capture verdict of the DNS fix-up); run outcomes; collection errors. The live collectors run in background threads — a scrape is a cache read, never an SSH session.	Prometheus scrapes the portal; dashboard <code>cat8000v-ipsec-overview</code>
MON-02	MUST	Alerts in the shared monitoring: <code>TunnelDown</code> , <code>HeadendUnreachable</code> , <code>HeadendCapacityExhausted</code> , <code>HeadendCpuHigh</code> , <code>InternetBreakoutDown</code> , <code>CertificateExpiringSoon</code> / <code>CertificateExpired</code> , <code>ConfigDrift</code> / <code>ConfigComplianceStale</code> , <code>LabTestsFailed</code> , <code>PortalDown</code> , and from the firewall log <code>FirewallDropBurst</code> / <code>FirewallDroppingPeerTraffic</code> .	<code>lab-portal/monitoring/prometheus/alerts.yml</code> , <code>vmlert/logs-alerts.yml</code>
MON-03	MUST	Firewall syslog lands in VictoriaLogs; the dashboard shows drops / accepts per firewall, drops by source, top dropped flows and the raw log; the portal queries the same store for the Firewalls page and a router's page.	suite 07 "The firewalls ship their firewall log to VictoriaLogs, and the portal..."
MON-04	SHOULD	Alerts only fire for a running lab (gated on the VM-running metric) so a powered-off lab is a normal state; dashboards are generated from code (<code>gen_dashboards.py</code>) and deployed with one script.	<code>monitoring/deploy.sh</code>

19. Verification: the Robot Framework suites

ID	Prio	Requirement	Acceptance criteria
TST-01	MUST	The whole service is proven by tests against the live lab: 01 management, 02 underlay, 03 IPsec VTI, 04 routing, 05 NaC compliance, 06 Nautobot model, 07 firewalls, 08 PKI, 09 hosts, 10 portal, 11 internet, 12 the DCI chain (incl. the twice-NAT, the DNS fix-up at scale and the packet capture that proves it) — 75 tests; every test reads the expected values from the intent / Nautobot, never from constants.	<code>./lab.sh test</code> ; <code>tests/resources/lab_vars.py</code>

TST-05	MUST	A suite waits for the portal's run queue before it starts (jobs execute one at a time; a suite started mid-job fails every test that needs the queue) and names the job it is waiting for; the portal's own test step is exempt, and <code>QUEUE_WAIT=0</code> skips the check. The scheduled Golden Config run skips its cycle while a job is running.	<code>tests/queue_state.py</code> , <code>tests/run.sh</code> ; <code>_golden_scheduler</code>
TST-02	MUST	Tests that change the lab do so for real and leave it as found : a renewal, a PSK ↔ certificate switch there and back, injected drift remediated and re-applied, queued runs cancelled — with teardowns.	suites 08 and 10
TST-03	MUST	Every run keeps pre / post configuration backups of every router and a diff of what the run changed under <code>results/<timestamp>/</code> with Robot's log and report; <code>results/latest</code> points at the last run; results are committed with the change they validate, together with the .pcap files of the DNS capture. Every run also produces report.pdf — an evidence report built from <code>output.xml</code> (<code>tests/report_pdf.py</code>): every suite and test with its result, its duration and the numbers it measured.	<code>tests/run.sh</code> , <code>capture_configs.py</code>
TST-04	SHOULD	Tests can be run from the portal (a <code>test</code> run, or at the end of any pipeline) and their report is parsed into the run (suites, tests, pass / fail, elapsed).	<code>parse_robot</code> ; <code>Run.tests</code>

20. Lab infrastructure and operations

ID	Prio	Requirement	Acceptance criteria
LAB-01	MUST	<code>lab.sh</code> is the one operations entry point: <code>up down status console ssh bootstrap wait log nac nautobot (onboard seed render vyos pki golden token) test intent webapp rename rebuild clean hosts</code> ; VMs are libvirt domains generated from <code>lab.conf</code> (node index → MACs, UDP socket links, console ports), with day-0 ISOs, license reload and RESTCONF wait on first boot.	README quick start
LAB-02	MUST	New routers, firewalls and hosts can be added without editing XML : the portal registers them in <code>lab.conf</code> , writes the day-0 config, defines and boots the VM; a spoke can be re-defined for a new NIC (re-home) and removed with its disk.	<code>spokes.add_to_lab_conf</code> / <code>write_day0</code> / <code>destroy_vm</code>
LAB-03	SHOULD	The portal runs as a user systemd service, the monitoring as Docker Compose on the NMS with one deploy script; both labs on the host never run their test suites concurrently.	<code>lab-webapp.service</code> , <code>monitoring/deploy.sh</code>

21. Documentation and demos

ID	Prio	Requirement	Acceptance criteria
----	------	-------------	---------------------

DOC-01	MUST	A README describing the topology, quick start, every portal feature with screenshots, the Nautobot model, the pipeline, the test suites and the file map; component READMEs for the portal, Nautobot scripts, NaC, tests and monitoring.	README.md , webapp/README.md , nautobot/README.md , ...
DOC-02	MUST	Generated documents kept current from the code: the workflows / data-flow / decision-tree PDF, this requirements specification, the screenshots (docs/screenshots.py) and an annotated demo video of the portal (webapp/demo/record.py , MP4 + GIF) plus a Nautobot / NaC data-flow demo.	docs/workflows.pdf , docs/requirements.pdf , webapp/demo/

22. Non-functional requirements

ID	Prio	Requirement	Measure
NFR-01	MUST	Idempotence: seed, render, firewall push, pki and apply can be re-run at any time and change nothing when nothing changed.	seed "0 changes", render --check , vyos --check , pki --check , plan exit code 0
NFR-02	MUST	Safety of change: nothing is pushed without a plan; applies are staged; verification steps fail the run rather than leave a half-authenticated tunnel; a run never starts on an invalid spec; runs are serialised.	§6, §8, §14
NFR-03	MUST	Traceability: every change carries a change ticket, a user and a run id; Nautobot records fingerprints, certificates, dates; Gitea holds every backup; the audit log every action; results directories every test run.	§13, §19
NFR-04	SHOULD	Responsiveness: pages render from caches within a second; a Golden Config run completes in about 20 s for 7 routers; a remediation in about 30 s; a re-apply in about 2 min; a scoped day-2 job (authentication switch, key rotation, certificate renewal — plans and applies targeted at the branch and its headends) in about 5 min; a job that renders, plans and applies the whole model in about 12 min with the DCI's scale set in it; the full test suite in about 55 min. Plans read in parallel (NAC_PLAN_PARALLELISM), applies stay serial.	observed run timings in the run records
NFR-05	SHOULD	Resilience: the portal survives Nautobot / Gitea / headend outages on read paths (errors reported in place, collectors degrade, metrics export an error series); interrupted runs are marked and resumable.	inventory_error , lab_inventory_error , lab_config_compliance_error
NFR-06	SHOULD	Footprint: LAN hosts are 256 MiB VMs; routers 4 GiB; the whole lab fits one 57 GiB host next to the SRv6 lab and the NMS.	memory check in spokes.validate before a VM is created
NFR-07	SHOULD	Usability: every value the wizard needs is suggested; every dialog previews what will change; every table links to the object's page and to Nautobot; dark mode; the UI degrades to the role.	§14, §15

23. Constraints, assumptions and known limits

- The iosxe provider's IKEv2 profile knows only pre-share authentication; rsa-sig, the certificate map and the trustpoint are pushed as a raw CLI template that depends on every native resource (so it always goes in the last apply stage). A CLI template's destroy does not revert lines; retiring a certificate removes the trustpoint and map, which makes IOS drop the dependent profile lines.
- IOS-XE shuts a VTI whose protection profile is swapped and refuses deleting an in-use profile or keyring — hence the staged apply and the post-apply verification.
- IOS-XE **appends** a route-map entry's description over RESTCONF instead of replacing it: a description whose text changes (it used to carry the internet-breakout preference rank, which a re-home changes) leaves the old line on the router and Golden Config reports drift. Rendered descriptions are therefore stable per object; the rank lives in `set local-preference`, which the provider does replace. Suite 11 asserts one description per route-map entry.
- Nautobot 3.2.4: a brand-new custom field drops values until it is saved a second time; the profile ↔ policy assignment endpoints 500 on POST (worked around); a location's description is limited to 255 characters.
- Golden Config compliance needs parseable backups: a multi-line banner breaks `hier_config` for that device.
- IOS-XE's CLI RPC rejects a very large payload (*bad length/size* at roughly two thousand lines): the renderer splits every CLI template into 400-line chunks, repeating the section header. A CLI template never removes the lines of a previous version, so shrinking a scale set means removing the old statements explicitly.
- The Alpine LAN hosts route 10.0.0.0/8 out of their OOB interface, so anything they must reach through the lab is addressed outside that range: the translated ranges live in 100.96.0.0/11 and the acquired company's real prefixes (10.128.0.0/14) are only ever reached through the NAT.
- VyOS static routes have no description leaf; the breakout uplink uses DHCP on the libvirt NAT network.
- Pre-shared keys are kept in the intent (a git-tracked file in the lab); a production deployment would move them to a secret store — the portal already treats them as secrets everywhere else.
- Customers are synthetic (deterministic fakes); the model, fields and filters are what a real CRM feed would fill.
- Both labs on the host share one Nautobot and one NMS; their test suites must not run concurrently.

24. Traceability matrix

Requirement area	Implemented in	Verified by
Network service (NET)	<code>nac/</code> data rendered by <code>nautobot/render_nac.py</code> ; day-0 templates in <code>lab.sh</code>	suites 01 – 05
Source of truth (SOT)	<code>nautobot/onboard.py</code> , <code>seed.py</code> , <code>golden_config.py</code> , <code>golden-config-templates/</code> , <code>intent.py</code>	suite 06
Intent (INT)	<code>nautobot/intent.py</code> , <code>webapp/schemas.py</code>	POST <code>/api/validate</code> ; suite 10
NaC pipeline (NAC)	<code>nac/</code> , <code>tools/nac_apply.py</code> , <code>webapp/app.py</code> (Run)	suites 05, 08, 10
Firewalls (FW)	<code>nautobot/render_vyos.py</code> , <code>webapp/firewalls.py</code>	suite 07

PKI (PKI)	<code>pki/ca.py</code> , <code>nautobot/pki.py</code> , <code>webapp/spokes.py</code> (auth / renewal / rotation plans)	suite 08, suite 03
Customers (CUST)	<code>nautobot/intent.py</code> (customer, design_pattern), <code>seed.py</code> (tenants, fields), <code>webapp/spokes.py</code> , <code>app.py</code> , <code>static/index.html</code>	suite 06
Interconnect, NAT and DNS (DCI)	<code>nautobot/intent.py</code> (nat , dns , dns_client , nat_scale), <code>render_nac.py</code> (templates, chunking), the Golden Config template, <code>seed.py</code> (namespace, tenants, peerings)	suite 12
Hosts (HOST)	<code>tools/host_cmd.py</code> , <code>lab.sh</code> hosts , <code>webapp/app.py</code> (lan_hosts)	suite 09
Internet (INET)	<code>render_vyos.py</code> , <code>render_nac.py</code> , config context internet	suite 11
Security (SEC)	<code>webapp/auth.py</code> , middleware in <code>app.py</code>	suite 10
Runs (RUN)	<code>webapp/app.py</code> (Run , run modes), <code>labportal</code> (RunRegistry), <code>webapp/spokes.py</code>	suites 08, 10
Pages (UI)	<code>webapp/static/index.html</code> , <code>webapp/inventory.py</code>	suite 10; screenshots; demo
Compliance (CMP)	<code>webapp/app.py</code> (compliance, scheduler, history, metrics), <code>nautobot/golden_config.py</code>	suites 05, 06, 10
API (API)	<code>webapp/app.py</code> , <code>schemas.py</code>	suite 10, Swagger
Monitoring (MON)	<code>lab-portal/monitoring/</code> , <code>webapp/app.py</code> (/metrics)	suite 07; Prometheus rules
Tests, lab, docs (TST, LAB, DOC)	<code>tests/</code> , <code>lab.sh</code> , <code>docs/</code> , <code>webapp/demo/</code>	the results directories